

COMS 4170 Software Testing Spring 2026

Chapter 4 – Test First

1

Reminder

- Homework 1 due tonight
- Can use slack day (up until Monday night)
- Only get one free slack day (across 4 assignments)

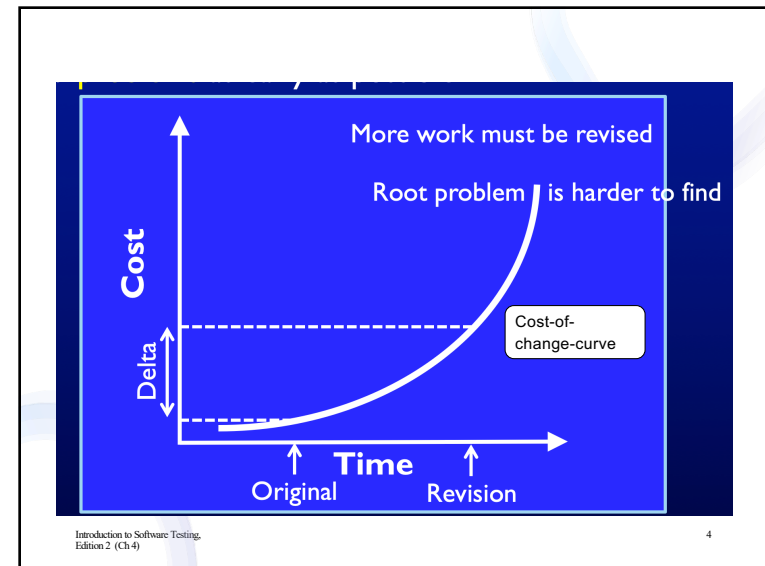
2

An Increased Emphasis on Testing

- Philosophy of traditional software development methods
 - Upfront analysis
 - Extensive modeling
 - Reveal problems as early as possible

Introduction to Software Testing,
Edition 2 (Ch 4)

3



4

Traditional Assumptions

- Modeling and analysis techniques can be effective in identifying problems and changes early in the software lifecycle
- The savings shown in the cost-of-change curve justify the cost of modeling and analysis techniques

5

But

- This assumes that requirements are correct and complete!

6

In Practice

- Requirements are changing
- Customers don't always know up front exactly what they want
- We have moved more to "agile" and iterative methods of development

7

Short Diversion....

- Software Lifecycles

8

Software Life Cycle

From inception of an idea for a product through

- Requirements gathering and analysis
- Architecture design and specification
- Coding and testing
- Delivery and deployment
- Maintenance and evolution
- Retirement

9

Software process model

- Attempt to organize the software life cycle by
 - Defining activities involved in software production
 - Order of activities and their relationships
- Goals of a software process
 - Standardization, predictability, productivity, product quality, ability to plan time and budget requirements

10

Code & Fix Lifecycle

The earliest approach

- 1 Write code
 - 2 Fix it to eliminate any errors that have been detected, or to enhance existing functionality, or to add new features
- Source of difficulties and deficiencies
 - Impossible to predict
 - Impossible to manage

11

11

Models are Needed

- Symptoms of inadequacy: the software crisis
 - Scheduled time and cost exceeded
 - User expectations not met
 - Poor quality
- The size and economic value of software applications require appropriate "process models"

12

12

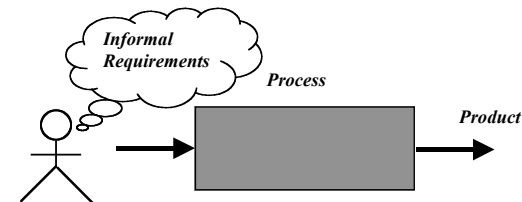
Process Model Goals (B. Boehm 1988)

"determine the order of stages involved in software development and evolution, and to establish the transition criteria for progressing from one stage to the next. These include completion criteria for the current stage plus choice criteria and entrance criteria for the next stage. Thus a process model addresses the following software project questions: What shall we do next? How long shall we continue to do it?"

13

13

Process as a Closed Box



14

14

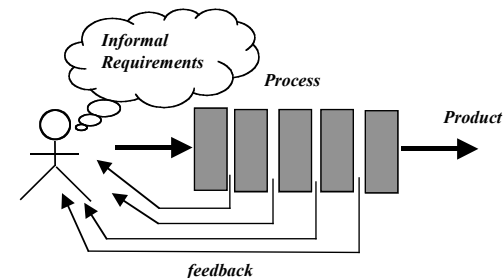
Problems

- The assumption is that requirements can be fully understood prior to development
- Interaction with the customer occurs only at the beginning (requirements) and end (after delivery)
- Unfortunately, the assumption almost never holds!

15

15

Process as an Open Box



16

16

Advantages

- Reduce risks by improving visibility
- Allows project changes as the project progresses
 - Based on feedback from the customer

17

17

Process Models

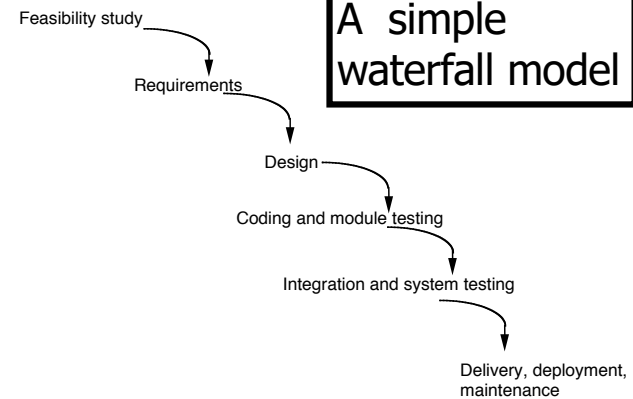
18

Waterfall Model(s)

- Invented in the late 1950s for large air defense systems
- Popularized in the 1970s
- Organizes activities in a sequential flow
- Has many variants - all share sequential flow style

19

19



20

Who Uses the Waterfall Model?

- Organizations adopting this model **standardize** the outputs of the various phases (**deliverables**)
- **Well understood** requirements, product
- Often **large companies**
- May also prescribe methods to follow in each phase
 - Organization of methods in frameworks often called *methodology*
- Example: Military Standard (MIL-STD-2167)

21

Critique of the Waterfall Model

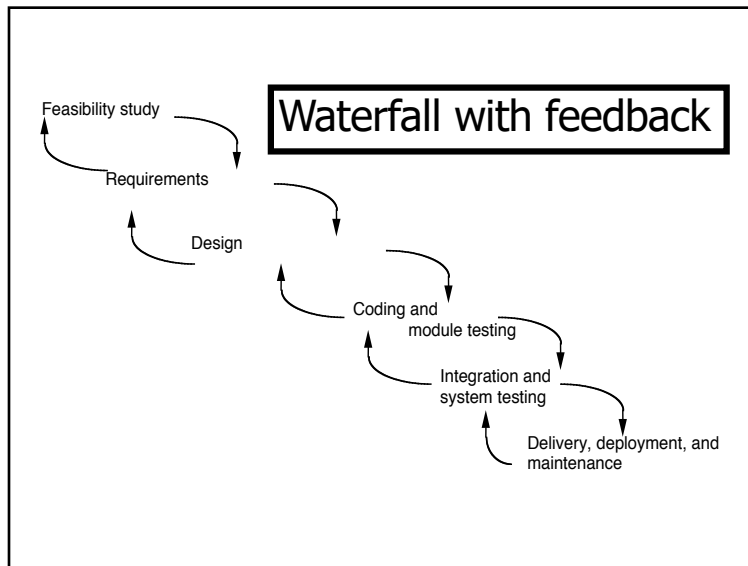
Positive:

- SE process **subject to discipline**, planning, and management
- Postpone implementation until **after understand objectives**

Negative:

- Linear, **rigid, monolithic**
no feedback
no parallelism
a single delivery date

22



23

However

- This still is **linear in practice**
- **Long periods of time** between initial requirements **gathering and evaluation of those requirements**

24

Agile Methods

- Dissatisfaction with the overheads involved in design methods led to the creation of agile methods. These methods:
 - Focus on the code rather than the design;
 - Are based on an iterative approach to software development;
 - Are intended to deliver working software quickly and evolve this quickly to meet changing requirements.
- Agile methods are probably best suited to small/medium-sized business systems or PC products.

25

Principles of Agile Methods

Principle	Description
Customer involvement	The customer should be closely involved throughout the development process. Their role is provide and prioritise new system requirements and to evaluate the iterations of the system.
Incremental delivery	The software is developed in increments with the customer specifying the requirements to be included in each increment.
People not process	The skills of the development team should be recognised and exploited. The team should be left to develop their own ways of working without prescriptive processes.
Embrace change	Expect the system requirements to change and design the system so that it can accommodate these changes.
Maintain simplicity	Focus on simplicity in both the software being developed and in the development process used. Wherever possible, actively work to eliminate complexity from the system.

26

26

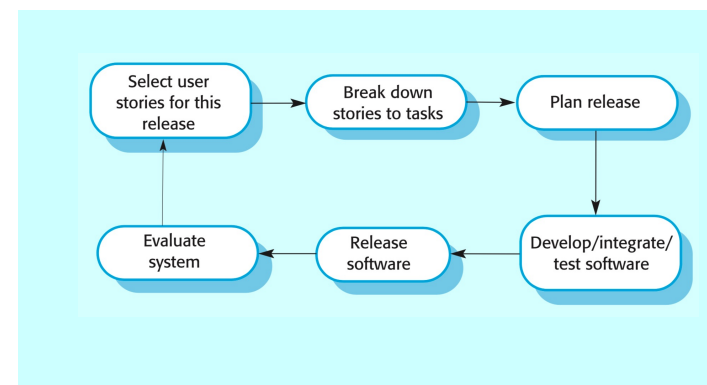
Extreme Programming (XP)

- Perhaps the best-known and most widely used agile method.
- Extreme Programming takes an 'extreme' approach to iterative development.
 - New versions may be built several times per day;
 - Increments are delivered to customers every 2 weeks;
 - All tests must be run for every build and the build is only accepted if tests run successfully!

27

27

The XP release cycle



28

28

Extreme Programming practices

Incremental planning	Requirements are recorded on Story Cards and the Stories to be included in a release are determined by the time available and their relative priority. The developers break these Stories into development 'Tasks'.
Small Releases	The minimal useful set of functionality that provides business value is developed first. Releases of the system are frequent and incrementally add functionality to the first release.
Simple Design	Enough design is carried out to meet the current requirements and no more.
Test first development	An automated unit test framework is used to write tests for a new piece of functionality before that functionality itself is implemented.
Refactoring	All developers are expected to refactor the code continuously as soon as possible code improvements are found. This keeps the code simple and maintainable.

29

29

Extreme Programming practices

Pair Programming	Developers work in pairs, checking each other's work and providing the support to always do a good job.
Collective Ownership	The pairs of developers work on all areas of the system, so that no islands of expertise develop and all the developers own all the code. Anyone can change anything.
Continuous Integration	As soon as work on a task is complete it is integrated into the whole system. After any such integration, all the unit tests in the system must pass.
Sustainable pace	Large amounts of over-time are not considered acceptable as the net effect is often to reduce code quality and medium term productivity
On-site Customer	A representative of the end-user of the system (the Customer) should be available full time for the use of the XP team. In an extreme programming process, the customer is a member of the development team and is responsible for bringing system requirements to the team for implementation.

30

30

XP and Agile Principles

- **Incremental development** is supported through **small, frequent** system releases.
- Customer involvement means full-time customer engagement with the team.
- **Pair programming**: collective ownership and a process that avoids long working hours.
- Change supported through **regular system releases**.
- Maintaining simplicity through **constant refactoring of code**.

31

31

Requirement scenarios

- In XP, user requirements are expressed as scenarios or **user stories**.
- These are **written on cards** and the development team break them down into **implementation tasks**. These tasks are the basis of schedule and cost estimates.
- The **customer chooses the stories for inclusion** in the next release based on their priorities and the schedule estimates.

32

Story Card for Document Downloading

Downloading and printing an article

First, you select the article that you want from a displayed list. You then have to tell the system how you will pay for it - this can either be through a subscription, through a company account or by credit card.

After this, you get a copyright form from the system to fill in and, when you have submitted this, the article you want is downloaded onto your computer.

You then choose a printer and a copy of the article is printed. You tell the system if printing has been successful.

If the article is a print-only article, you can't keep the PDF version so it is automatically deleted from your computer.

33

33

XP and change

- Conventional wisdom in software engineering is to **design for change**. It is worth spending time and effort anticipating changes as this reduces costs later in the life cycle.
- XP, however, maintains that this is **not worthwhile as changes cannot be reliably anticipated**.
- Rather, it proposes **constant code improvement** (refactoring) to make changes easier when they have to be implemented.

34

34

Testing in XP

- **Test-first development**.
- Incremental test development from scenarios.
- **User involvement** in test development and validation.
- **Automated test harnesses** are used to run all component tests each time that a new release is built.

35

35

Task Cards for Document Downloading

Task 1: Implement principal workflow

Task 2: Implement article catalog and selection

Task 3: Implement payment collection

Payment may be made in 3 different ways. The user selects which way they wish to pay. If the user has a library subscription, then they can input the subscriber key which should be checked by the system. Alternatively, they can input an organisational account number. If this is valid, a debit of the cost of the article is posted to this account. Finally, they may input a 16 digit credit card number and expiry date. This should be checked for validity and, if valid a debit is posted to that credit card account.

36

36

Test Case Description

Test 4: Test credit card validity

Input:

A string representing the credit card number and two integers representing the month and year when the card expires

Tests:

Check that all bytes in the string are digits

Check that the month lies between 1 and 12 and the year is greater than or equal to the current year.

Using the first 4 digits of the credit card number, check that the card issuer is valid by looking up the card issuer table. Check credit card validity by submitting the card number and expiry date information to the card issuer

Output:

OK or error message indicating that the card is invalid

37

37

Test-first development

- Writing **tests before code** clarifies the requirements to be implemented.
- **Tests are written as programs rather than data** so that they can be executed automatically. The test includes a check that it has executed correctly.
- All **previous and new tests are automatically** run when new functionality is added. Thus checking that the new functionality has not introduced errors.

38

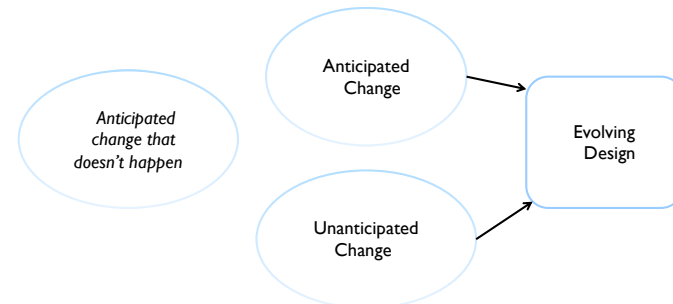
38

Back to
Our
Text....

39

Supporting Evolutionary Design

Traditional design advice says to anticipate changes
Designers often anticipate changes that don't happen



Both anticipated and unanticipated changes affect design

Introduction to Software Testing, Edition 2 (Ch 4)

© Ammann & Offutt

40

Important Part of Process

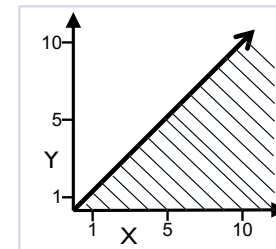
- **Test Harness**
 - Automated tests plus
 - The process by which the execution of tests is managed to provide critical feedback to developers

41

The Test Harness as Guardian What is Correctness ?

Traditional Correctness
(Universal)

$$\forall x, y, x \geq y$$



Agile Correctness
(Existential)

```
{ (1, 1) → T
  (1, 0) → T
  (0, 1) → F
  (10, 5) → T
  (10, 12) → F }
```

Introduction to Software Testing, Edition 2 (Ch 4)

© Ammann & Offutt

42

A Limited View of Correctness

- In traditional methods, we try to define all correct behavior completely, at the beginning
 - What is correctness?
 - Does "correctness" mean anything in large engineering products?
 - People are **not very good** at completely defining correctness

Introduction to Software Testing,
Edition 2 (Ch 4)

© Ammann & Offutt

43

A Limited View of Correctness

- In agile methods, we redefine correctness to be relative to a specific set of tests
 - If the software behaves correctly on the tests, it is "correct"?
 - Instead of defining all behaviors, we demonstrate some behaviors
 - Mathematicians may be disappointed at the lack of completeness

Introduction to Software Testing,
Edition 2 (Ch 4)

© Ammann & Offutt

44

Test Harnesses Verify Correctness

A *test harness* runs all automated tests efficiently and reports results to the developers

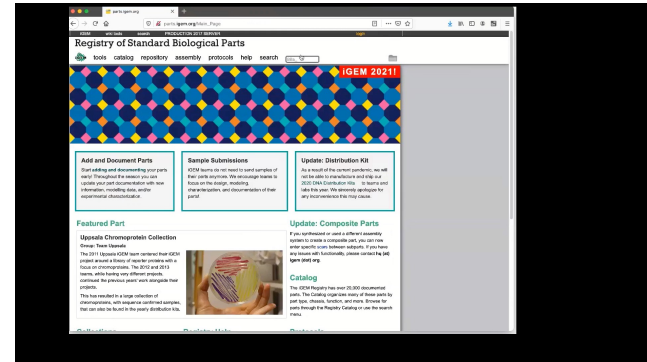
- Tests **must be automated**
 - Test automation is a prerequisite to test driven development
- Every test must include a **test oracle** that can evaluate whether that test executed correctly
- The **tests replace the requirements**
- Tests **must be high quality** and **must run quickly**
- We **run tests every time we make a change** to the software
 - continuous integration

Introduction to Software Testing, Edition 2 (Ch 4)

© Ammann & Offutt

45

Web Test Automation Coming Soon...



46

Continuous Integration

- Agile methods work best when the current version of the software can be run against all tests at any time

A *continuous integration server* rebuilds the system, returns, and re-verifies tests whenever *any* update is checked into the repository

- Mistakes are caught earlier
- Other developers are aware of changes early
- The rebuild and re-verify must happen as soon as possible

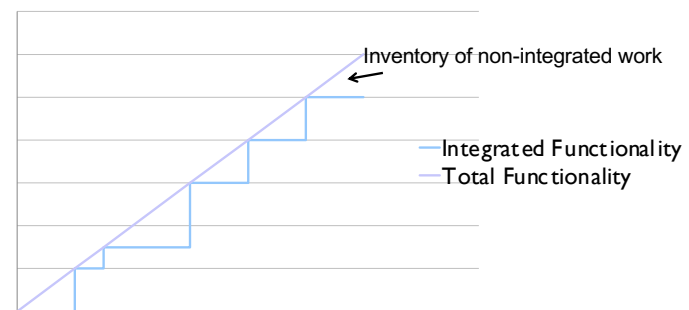
A *continuous integration server* doesn't just run tests, it decides if a modified system is still correct

Introduction to Software Testing, Edition 2 (Ch 4)

© Ammann & Offutt

47

Continuous Integration Reduces Risk



Non-integrated functionality is dangerous!

Introduction to Software Testing, Edition 2 (Ch 4)

© Ammann & Offutt

48

Example CI Tools

- Bitbucket Pipelines
- Jenkins
- AWS CodePipeline
- Azure Pipelines
- GitLab (pipelines)
- Atlassian Bamboo
- GitHub Actions
- Google cloud
-

49

Testing at Google

- <https://research.google/pubs/pub45880/>

Google

The State of Continuous Integration
Testing @Google

By: John Micco - jmicco@google.com
投稿者: ジョン・ミッコ

50

Testing Scale at Google

- 4.2 million individual tests running continuously
 - Testing runs before and after code submission
- 150 million test executions / day (averaging 35 runs / test / day)
- Distributed using internal version of bazel.io to a large compute farm
- Almost all testing is automated - no time for Quality Assurance
- 13,000+ individual project teams - all submitting to one [branch](#)
- Drives continuous delivery for Google
- 99% of all test executions pass

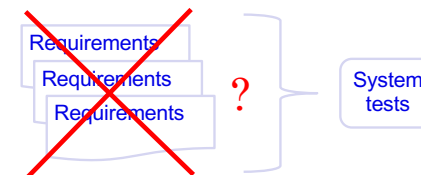


Google

51

System Tests in Agile Methods

Traditional testers often design
system tests from requirements



But ... what if there are no
traditional requirements
documents ?

Introduction to Software Testing, Edition 2 (Ch 4)

© Ammann & Offutt

52

A user story is a few sentences that captures what a user will do with the software

Withdraw money from checking account	Agent sees a list of today's interview applicants
Support technician sees customer's history on demand	

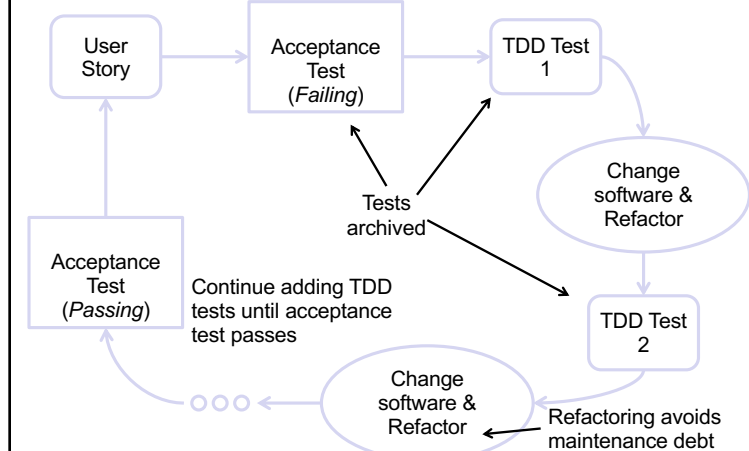
- In the language of the end user
- Usually small in scale with few details
- Not archived

Introduction to Software Testing, Edition 2 (Ch 4)

© Ammann & Offutt

53

Acceptance Tests in Agile Methods



Introduction to Software Testing, Edition 2 (Ch 4) © Ammann & Offutt

54

Adding Tests to Existing Systems

- Most of today's software is legacy
 - No legacy tests
 - Legacy requirements hopelessly outdated
 - Designs, if they were ever written down, lost

How do we apply TDD to legacy software with no tests?

- Create an entire new test set? — too expensive!
- Give up? — a mixed project is unmanageable

Introduction to Software Testing, Edition 2 (Ch 4)

© Ammann & Offutt

55

Incremental TDD

- When a **change is made**, add **TDD tests for just that change**
 - **Refactor**
- As the **project proceeds**, the **collection of TDD tests continues to grow**
- Eventually the software will have strong TDD tests

Introduction to Software Testing, Edition 2 (Ch 4)

© Ammann & Offutt

56

The Testing Shortfall

- Do TDD tests (acceptance or otherwise) test the software well?
 - Do the tests achieve **good coverage** on the code?
 - Do the **tests find most** of the faults?
 - ***If the software passes, should management feel confident the software is reliable?***

No

Introduction to Software Testing, Edition 2 (Ch 4)

© Ammann & Offutt

57

Why Not?

- Most agile tests focus on “*happy paths*”
 - What should happen under normal use
- They often miss things like
 - Confused-user paths
 - Creative-user paths
 - Malicious-user paths

The agile methods literature does not give much guidance

Introduction to Software Testing, Edition 2 (Ch 4)

© Ammann & Offutt

58

What Should Testers Do?

Introduction to Software Testing, Edition 2 (Ch 4)

© Ammann & Offutt

59

59

Design Good Tests

Use a human-based approach

- Create additional user stories that describe non-happy paths
- How do you know when you're finished?
- Some people are very good at this, some are bad, and it's hard to teach

Part 2 of
book ...

Use modeling and criteria

- Model the input domain to design tests
- Model software behavior with graphs, logic, or grammars
- A built-in sense of completion
- Much easier to teach—engineering
- Requires discrete math knowledge

Introduction to Software Testing, Edition 2 (Ch 4)

© Ammann & Offutt

60

Summary

- More companies are putting testing first
- This can dramatically decrease cost and increase quality
- A different view of “correctness”
 - Restricted but practical
- Embraces evolutionary design
- TDD is definitely not test automation
 - Test automation is a prerequisite to TDD
- Agile tests aren’t enough